

VISTA PARA EL EQUIPO DE DESARROLLO — ARQUITECTURA & PATRONES TO-BE
Sistema de Gestión de Hacienda: Guía Técnica de Modificación Segura

🎯 Para quién: El ingeniero que entra al equipo dentro de 6 meses y debe hacer un cambio sin romper nada y sin haber asistido a ninguna reunión previa. Responde: qué patrones hay, dónde se ensamblan, qué reglas no romper, la deuda técnica y la matriz exacta de archivos a tocar.

1. Mapa de Patrones y Punto de Ensamblaje

Patrón	ID	Ubicación	Propósito e Interacción
Factory Method	P-05	Bib_Hacienda/Creacion/	Instancia Ternero, Cebon o Novillo según el tipo de potrero (CatalogoCreadoresRes).
Strategy	P-02/03	Bib_Hacienda/Estrategias/	Encapsula reglas por tipo de res (pesos min/venta y topes vacunas en SelectorEstrategiaTipoRes).
Observer	P-01	Bib_Hacienda/Eventos/	Notifica desnutrición, llenado y vencimiento a RecolectorMensajesEventos (suscripción única).
Builder	P-04	Bib_Hacienda/Construccion/	Construye vacunas Bacterianas/Vivas centralizando validaciones de lotes, fechas y nombres.
Repository + DIP	ADR-03/07	p_mvchacienda/Servicios/R epositorios/	Aísla el almacenamiento físico mediante IAlmacenamiento<T> (preparado para migrar a SQL).
Segregación ISP	ADR-04	Bib_Hacienda/Clases/Valida ciones/	Validadores específicos por entidad (IValidador<T>), eliminando NotImplementedException.
Composición OCP	ADR-08	Bib_Hacienda/Clases/	Agrega Chip y HistoriaClinica a Res por composición sin alterar la jerarquía de herencia.

📍 Dónde se ensambla: En p_mvchacienda/Program.cs (IoC Container: registra Singletons de IAlmacenamiento, Repositorios, IValidador, PersistenciaService y la instancia Hacienda) y en los puntos de registro del dominio (CatalogoCreadoresRes, SelectorEstrategiaTipoRes y constructores de Hacienda/Potrero).

2. El Artefacto Central: Guía de Dónde Tocar (Matriz de Cambio Seguro)

Tipo de Cambio Previsible	¿Qué CREAR? [NEW]	¿Qué MODIFICAR? [MODIFY]	¿Qué NO TOCAR? [DO NOT TOUCH]
ANEXO B.1: Nuevo Tipo de Res / Potrero (Ej. Toro / Vaca Lechera)	• Clases/Toro.cs (subclase Res) • Creacion/CreadorToro.cs (hereda CreadorRes) • Estrategias/EstrategiaToro.cs (IEstrategiaTipoRes)	• Potrero.cs: Añadir enum l_tipos_potreros. • CatalogoCreadoresRes.cs: Registrar fábrica. • SelectorEstrategiaTipoRes.cs: Añadir estrategia. • Reglas (ReglaRes / ReglaVacuna) y Vistas Razor.	🚫 Hacienda.cs (anadir_res_potrero) 🚫 Potrero.cs (anadir_res) 🚫 CreadorRes base 🚫 Publishers Observer 🚫 Controladores MVC
ANEXO B.2: Nuevo Tipo de Vacuna (Ej. Recombinante / Viral)	• Clases/Recombinante.cs (hereda Vacuna) • Métodos en	• VacunaBuilder.Construir(): Instanciar tipo. • ICreacionVacuna /	🚫 Vacuna.cs base 🚫 VacunaBuilder.ValidarComu

	VacunaBuilder.cs (ComoRecombinante)	Hacienda: Sobrecarga crear_vacuna. • IEstrategiaTipoRes (si tiene nuevos topes). • VacunaController y vistas Razor de Vacuna.	n() ❌ Lógica lotes en Hacienda.cs ❌ PublisherVacunaVencida.cs ❌ Repositorio Vacunas
ANEXO B.3: Módulo Transversal IoT (Ej. Chip GPS / Historia Clínica)	• Clases/Chip.cs (objeto de valor) • GeolocalizacionService / IGeolocalizacionService • Vistas/Controlador específico	• Res.cs: Propiedad public Chip Chip { get; set; }. • Program.cs: Registrar servicio en IoC. • ResRepositoryTxt.cs: Serializar campo.	❌ Jerarquía herencia de Res ❌ Constructores Ternero/Cebon/Novillo ❌ CreadorRes y fábricas ❌ Potrero.cs
PREV-04: Migración a Base de Datos SQL (Ej. SQL Server / EF Core)	• AlmacenamientoSql.cs (IAlmacenamiento<T>) o *RepositorySql.cs • DbContext y migraciones EF Core	• Program.cs: Cambiar AlmacenamientoTxt por AlmacenamientoSql en contenedor IoC. • appsettings.json: Cadena de conexión.	❌ Todo Bib_Hacienda (Dominio 100% puro) ❌ PotreroService, ResService, etc. ❌ Interfaces repositorios ❌ Controladores y vistas
PREV-05: Nuevo Evento / Alerta Reactiva (Ej. Alerta sobrepeso / cuarentena)	• Eventos/PublisherSobrepeso.cs (delegado y evento evt_sobrepeso)	• Hacienda.cs o Potrero.cs: Instanciar en constructor, suscribir en SuscribirseEventos(), desuscribir y disparar en operación.	❌ RecolectorMensajesEventos.cs ❌ Controladores MVC ❌ Res.cs / Potrero.cs (no tocan UI) ❌ Repositorios
PREV-06: Modificar Regla de Validación (Ej. Formato ID / peso válido)	• (Si es nueva entidad): ValidadorNueva : IValidador<NuevaEntidad>	• ValidadorPotrero.cs / ValidadorRes.cs: Ajustar método Validar(entidad). • Program.cs: Registrar si es validador nuevo.	❌ Interfaz genérica IValidador<T> ❌ PersistenciaService.cs ❌ Controladores y repositorios

3. Reglas Invariantes, Deuda Técnica y Verificación

Reglas Invariantes (NO Romper)	Deuda Técnica Pendiente	Checklist de Verificación (5 min)
1. Cero switch/if sobre tipos de res: Todo polimorfismo va en Strategy o Factory (OCP). 2. Suscripción SOLO en constructores: El recolector se limpia en cada acción; no hacer += en métodos. 3. Vacunas SOLO con VacunaBuilder: Centraliza unicidad de lotes y fechas. 4. Capa Web aislada: Controller → Service → Dominio/Persistencia → Repositorio → IAlmacenamiento (DIP). 5. Validadores segregados (ISP): Usar IValidador<T> por entidad.	1. Concurrencia TXT: AlmacenamientoTxt reescribe archivo completo. Pendiente migrar a SQL o ReaderWriterLockSlim. 2. Sincronización memoria Hacienda: Listas en memoria sincronizadas manualmente. Pendiente Unit of Work. 3. Interceptores AOP: Acoplados a HttpContext.Items. Pendiente filtros nativos ASP.NET Core. 4. Consistencia en cascada: Al vender res, limpiar históricos en otros archivos con IDs inmutables.	1. Ubica tu cambio en la matriz central. 2. Crea los archivos [NEW] y edita solo los [MODIFY]. 3. Comprueba no haber tocado los [DO NOT TOUCH]. 4. Ejecuta la suite de verificación automatizada: dotnet run --project .\VerificacionActividad4\VerificacionActividad4.csproj 5. Resultado esperado: 12 coinciden, 0 difieren.